



Building Semantic Product Search in OpenSearch: Ingestion, Chunking, and Hybrid Search

⚙ Status Drafts

Building Semantic Product Search in OpenSearch: Ingestion, Chunking, and Hybrid Search

This tutorial walks through preparing an e-commerce product catalog for semantic search in OpenSearch. You'll learn how vector search works, how to generate embeddings at index time with an ingest pipeline, when chunking helps, and how the indexing and query sides fit together into hybrid search.

1. What is vector search?

Vector search converts text into an embedding — a list of numbers that represents meaning:

```
"waterproof running jacket"
  ↓
Embedding model
  ↓
[0.023, -0.114, 0.882, ...]
  ↓
Vector similarity search
```

Because the match is on meaning rather than exact words, a query like:

```
"warm clothes for jogging in rain"
```

can return:

```
Men's Waterproof Running Jacket
```

even though the product text never contains "warm clothes" or "jogging in rain". This is what makes vector search strong for natural-language queries.

2. What is an ingest pipeline?

An **index** is where OpenSearch stores a collection of related documents — roughly what a table is in a relational database, with each document playing the role of a row. So a `products` index holds all your product documents.

An ingest pipeline is a processing workflow that runs **before** a document is stored in an index. Its processors run in sequence:

```
Raw document
  ↓
Ingest Pipeline
  ↓
Processor 1 (lowercase)
  ↓
Processor 2 (html_strip)
  ↓
Processor 3 (text_embedding)
  ↓
Processed document
  ↓
OpenSearch Index
```

Think of it as middleware between your source product data and the index.

Step 1: Start with your raw product

```
{
  "name": "Heavy Cotton T-Shirt",
  "description": "Classic cotton t-shirt for promotional printing",
  "brand": "Acme Apparel"
}
```

Step 2: Create the pipeline

```
PUT /_ingest/pipeline/product-ingest-pipeline
{
  "description": "Prepare products for semantic search",
  "processors": [
    {
      "text_embedding": {
        "model_id": "<model-id>",
        "field_map": {
          "description": "description_embedding"
        }
      }
    }
  ]
}
```

The `text_embedding` processor takes a text field, sends it through the configured ML model, and writes the resulting vector into a new field.

Step 3: Index through the pipeline

```
POST /products/_doc?pipeline=product-ingest-pipeline
```

OpenSearch stores the document with the embedding attached:

```
{
  "name": "Heavy Cotton T-Shirt",
  "description": "Classic cotton t-shirt for promotional printing",
  "brand": "Acme Apparel",
  "description_embedding": [0.023, -0.127, 0.442, ...]
}
```

Two processors matter most for AI search: `text_embedding` (shown above) and `text_chunking`, covered next.

3. What is chunking, and do you need it?

Chunking splits a large piece of text into smaller pieces before embedding it. Instead of embedding one block:

```
"Heavy Cotton T-Shirt.
100% cotton.
Available in 30 colors.
Suitable for screen printing.
Adult sizes S-5XL."
```

you split it:

```
Chunk 1: "Heavy Cotton T-Shirt. 100% cotton."
Chunk 2: "Available in 30 colors. Suitable for screen printing."
Chunk 3: "Adult sizes S-5XL."
```

Why does chunking improve precision?

Embedding models work best when each piece of text carries a focused meaning. If a single document mixes description, material, sizing, printing methods, shipping, and care instructions into one vector, a query like `"shirt suitable for screen printing"` gets diluted across all of it. With chunks, the query can match strongly against just:

```
"Suitable for screen printing and heat transfer decoration."
```

How big should a chunk be?

Chunks are measured in tokens, characters, or sentences. Hard boundaries can cut a sentence's context in half, so chunks usually overlap:

Without overlap	With 50-token overlap
Chunk 1: 1–300	Chunk 1: 1–300
Chunk 2: 301–600	Chunk 2: 251–550
Chunk 3: 601–900	Chunk 3: 501–800

How do you chunk in OpenSearch?

Chain the chunking processor before the embedding processor:

```
description
  ↓
text_chunking
  ↓
description_chunk
  ↓
text_embedding
  ↓
chunk_embeddings
```

The chunking step turns one field into an array:

```
{
  "description_chunk": [
    "chunk one...",
    "chunk two...",
    "chunk three..."
  ]
}
```

So when should you chunk?

Embedding converts text into a vector. Chunking decides how much text goes into each embedding.

For a typical product catalog, **start without chunking**. If your searchable text is short:

```
Heavy Cotton T-Shirt
Acme Apparel
100% cotton
classic fit
screen printing compatible
men's apparel
```

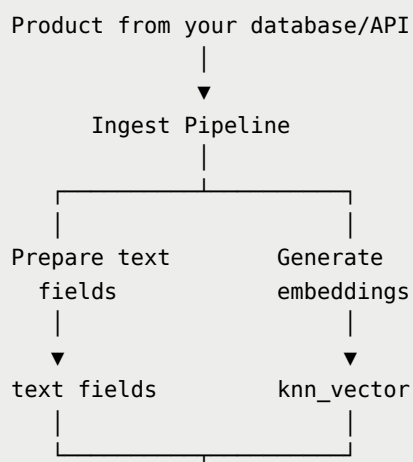
build one `search_text` field, generate one embedding, and store one vector. Chunking here only adds complexity.

Introduce chunking only when a product accumulates enough text — long description, features, technical specs, decoration instructions, care instructions, supplier notes, FAQs — that a single embedding loses semantic specificity.

4. How does this fit into hybrid search?

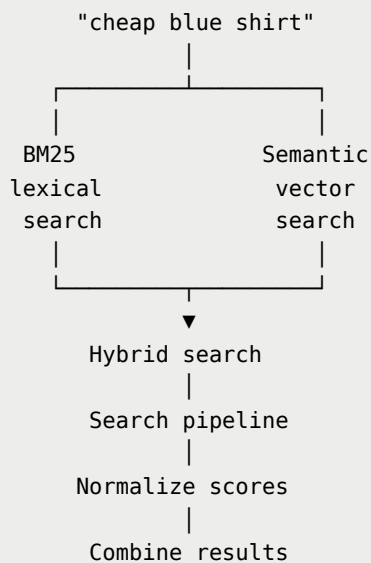
Keep the two sides of the system separate in your mind.

Indexing side — the ingest pipeline prepares text fields and generates embeddings:



▼
products_v1

Search side — a search pipeline normalizes and combines lexical and semantic results:



The distinction to remember:

Ingest pipeline = processing documents BEFORE indexing
Search pipeline = processing results DURING querying

Summary

1. Vector search matches meaning, not keywords.
2. An ingest pipeline generates embeddings automatically at index time.
3. Chunking is optional — add it only when documents get large.
4. Hybrid search combines BM25 and vector results through a search pipeline.